

JobPing / Match Point Project — Build Plan (Scratch Rebuild)

PROJECT OVERVIEW

Full-stack AI-powered job search automation SaaS. The app scans a user's Gmail inbox, uses a single merged OpenAI call to classify emails (job advert, interview, offer, rejection) AND score job adverts against the user's uploaded CV(s) in one request, then sends a Telegram notification for any job scoring 7/10 or above. A later phase adds a conversational CV refinement assistant.

Name: TBD (candidates explored: Match Point, Job Ping, Job Echo, Scanly, Mail Pulse, Mailpin — domain not yet secured, revisit later)

Goals:

1. Learning project — hands-on Azure deployment experience
2. Personal utility — stop missing good job alerts buried in Gmail
3. Keep running cost as close to \$0 as possible

KEY CHANGES FROM PREVIOUS VERSION (Mail_Agent_V1)

- AI provider: Groq LLaMA 3.3-70B -> OpenAI (model TBD, likely gpt-4o-mini for cost)
- Classification + CV scoring: previously 2 separate calls -> merged into ONE call returning structured JSON (classification + score + reasoning + best-matching CV)
- CV handling: single CV -> MULTIPLE CVs uploaded, no manual tagging; AI compares job against all CVs and returns best match + score
- CV refinement: new feature — conversational AI chat, scoped to one CV + one job advert, user-initiated ("refine for this job" button), AI proactively leads with gap analysis, text-based editing
- Background jobs: Celery + Redis + Celery Beat -> Azure Functions (timer trigger), removes need for a constantly-running worker
- Deployment target: Azure (App Service for backend/frontend, Azure Functions for scheduled scanning)
- Domain: custom domain to be purchased (decision deferred)
- Notification threshold unchanged: score > 7/10 triggers Telegram ping

TECH STACK

Backend: FastAPI (Python)

Frontend: React + Vite (JavaScript)

Database: Supabase (PostgreSQL)

AI: OpenAI API (model TBD — gpt-4o-mini candidate)

Scheduling: Azure Functions (timer trigger) — replaces Celery/Redis

Notifications: Telegram Bot API (primary), browser push optional/later

Auth: JWT (python-jose + passlib bcrypt==4.0.1)

Gmail: Google OAuth2 (Web App credential)

Deployment: Azure App Service (backend + frontend) + Azure Functions

Domain: Custom domain (TBD), HTTPS

BUILD PHASES (REORDERED — Telegram moved early for fast feedback loop)

Phase 0 — Foundation

- Folder structure, Git init, virtual environment
- FastAPI entry point with a health-check endpoint
- Logging setup (Loguru), .env file and .env.example

Phase 1 — Backend Core (Auth + Models)

- SQLAlchemy/Pydantic models: User, Email, Job, CV, Application, ScanLog
- JWT authentication: signup, login, logout
- get_current_user dependency protecting routes
- HTTPBearer scheme (plays nicer with Swagger UI than OAuth2PasswordBearer)

Phase 2 — Supabase Database

- Connect via Supabase Python client over HTTPS (avoids direct Postgres port 5432 issues some networks block)
- Tables: users, emails, jobs, cvs, applications, scan_logs, gmail_tokens
- Test all auth endpoints against real Supabase tables

Phase 3 — Gmail Scanner + Merged OpenAI Classification/Scoring

- gmail_scanner.py — fetch emails via Gmail API (manual trigger for now, e.g. a script or protected endpoint — no Azure Function yet)
- Single OpenAI call per email returning structured JSON: is_job_advert, category (advert/interview/offer/rejection), match_score (1-10), best_matching_cv_id, reasoning
- Compares job advert against ALL uploaded CVs in one pass, returns best match
- scanner.py — orchestrator tying fetch -> classify -> score together
- TEST AGAINST REAL GMAIL ACCOUNT HERE — this is the core logic validation step before building anything else

Phase 4 — Telegram Notifications (moved early deliberately)

- Telegram Bot API integration
- send_telegram(), format_job_notification()
- Fires only when match_score > 7
- Silent if no high-match jobs found
- GOAL: end-to-end "it works" moment — scan Gmail, get a real Telegram ping for a real job

Phase 5 — React Frontend

- Pages: Login, Signup, Dashboard, Emails, Jobs, CVUpload (multi-CV support), Settings
- Axios service layer with JWT auto-attach interceptor
- AuthContext + ProtectedRoute
- CORS: allow both localhost:5173 and 127.0.0.1:5173

Phase 6 — Per-User Gmail OAuth

- /gmail/connect, /gmail/callback, /gmail/status, /gmail/disconnect
- gmail_tokens table stores per-user credentials
- Manual requests.post() token exchange (avoids PKCE error from google-auth-oauthlib Flow object)
- Web Application credential type, not Desktop app

Phase 7 — CV Refinement Chat

- User clicks "refine for this job" on a scored job
- Chat interface scoped to: one CV + one job advert as context
- AI proactively opens with gap analysis ("you scored 9/10, here's what to tighten up")
- Text-based editing, conversational back-and-forth, not a one-shot rewrite

- Output: refined CV text the user can copy/export

Phase 8 — Azure Functions (replacing manual trigger / Celery)

- Wrap Phase 3 scan logic in an Azure Function with a timer trigger
- Suggested interval: every 30 minutes (matches old SCAN_INTERVAL_MINUTES)
- Test locally first with Azure Functions Core Tools before deploying
- Consumption plan — pay only per execution, generous free tier (~1M executions/month)

Phase 9 — Azure Deployment + Domain

- Backend: Azure App Service (consider F1 free tier limitations — sleeps when idle; weigh against a low-cost basic tier)
- Frontend: Azure Static Web Apps (has a solid free tier)
- Keep Supabase as-is rather than migrating to Azure-managed Postgres (cost control)
- Enable Row Level Security in Supabase before going live
- Purchase and attach custom domain + HTTPS
- GitHub Actions CI/CD pipeline

OPEN DECISIONS / TO REVISIT

- Final app name + domain (squatted/premium on several tried: matchpoint.com, jobping.com/.io, mailpulse — revisit with a fresh registrar search, consider made-up/blended words to dodge squatters)
- Exact OpenAI model choice (cost vs. reasoning quality for CV scoring)
- Azure App Service tier for backend (free F1 vs. low-cost Basic, given F1 sleeps on idle)
- Whether browser push notifications are worth keeping or if Telegram alone is sufficient for v1

KNOWN ISSUES CARRIED OVER FROM PREVIOUS BUILD (keep in mind)

1. bcrypt must be ==4.0.1 (newer versions break passlib)
2. CORS must include both localhost:5173 AND 127.0.0.1:5173
3. Gmail OAuth: use Web Application credential + manual requests.post() token exchange (Flow object triggers PKCE error)
4. Supabase direct Postgres connection (port 5432) can be blocked by some networks — use the HTTPS client instead
5. token.json / credentials.json belong in backend/ folder, not project root
6. If running from backend/ folder, set PYTHONPATH=.

NEXT STEP

Continue in a text-based chat for actual code, file-by-file, phase-by-phase, starting with Phase 0 (foundation) and Phase 1 (backend core + auth).